任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关？

以用户登录页面为例，用户需要在页面输入账号、密码，系统要完成登录验证，就必须拿着这些输入的内容，去数据库的用户表中查询匹配的用户数据，确认账号存在、密码正确。页面输入是数据库查询的条件来源，没有用户输入的条件，就无法从数据库里筛选出对应的目标数据，页面的登录功能也就无法实现，所以页面输入和数据库查询是直接关联的。

2. 一个最小业务链路是什么？

输入条件（用户在登录页输入账号、密码）→ 数据库查询（执行 SELECT * FROM 用户表 WHERE 账号=用户输入的账号 AND 密码=用户输入的密码）→ 返回结果（匹配到用户则返回登录成功，否则返回登录失败）

3. 用户输入是如何进入查询条件的？

用户在输入框里填的内容，会被前端通过 HTTP 请求（如 POST）发给后端。

后端代码（如 PHP、Java、Python）取出这些值，直接或经过拼接，放进 SQL 查询语句的 WHERE 子句中，成为查询条件的一部分。

4. 为什么这为 SQL 注入创造了可能性

如果后端没有对用户输入做安全过滤、校验，攻击者就可以构造特殊的恶意输入。

比如账号输入 zhangsan' OR '1'='1，密码随意输入，拼接后的 SQL 就变成：

SELECT * FROM 用户表 WHERE 账号='zhangsan' OR '1'='1' AND 密码='xxx'

因为'1'='1'是恒成立的条件，这条 SQL 会绕过密码验证，直接返回所有用户数据，攻击者就实现了 SQL 注入。

任务 2：DVWA SQL Injection Low 现象记录

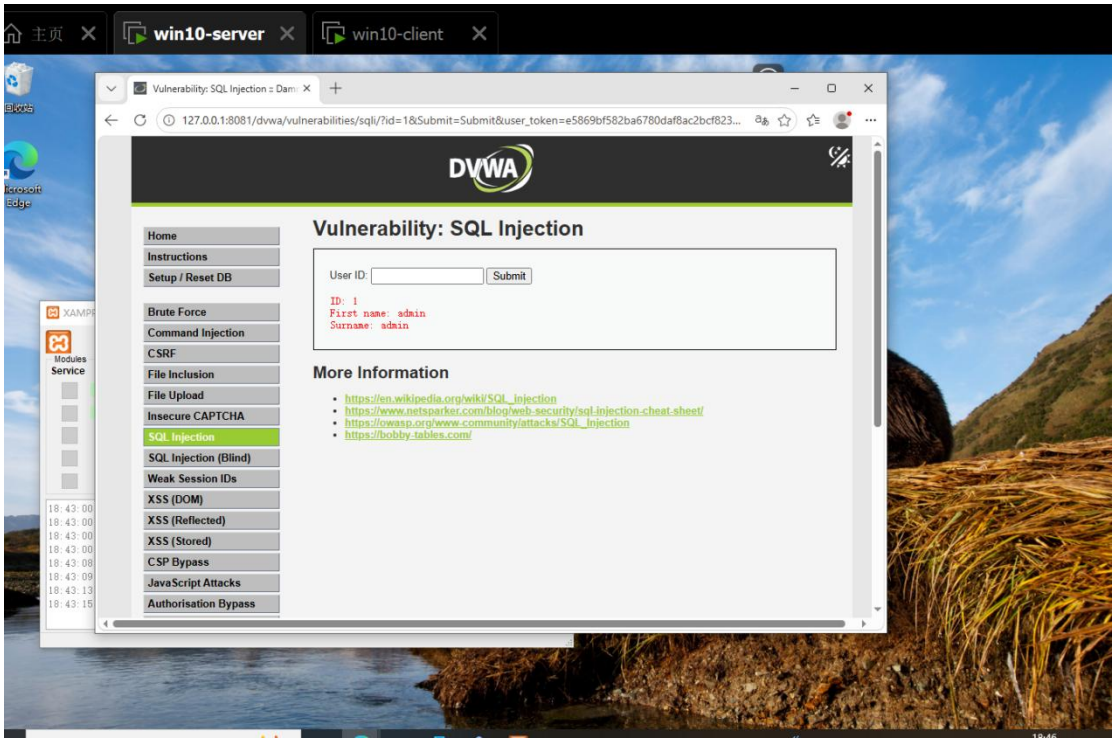
1. 实验页面或实验目标

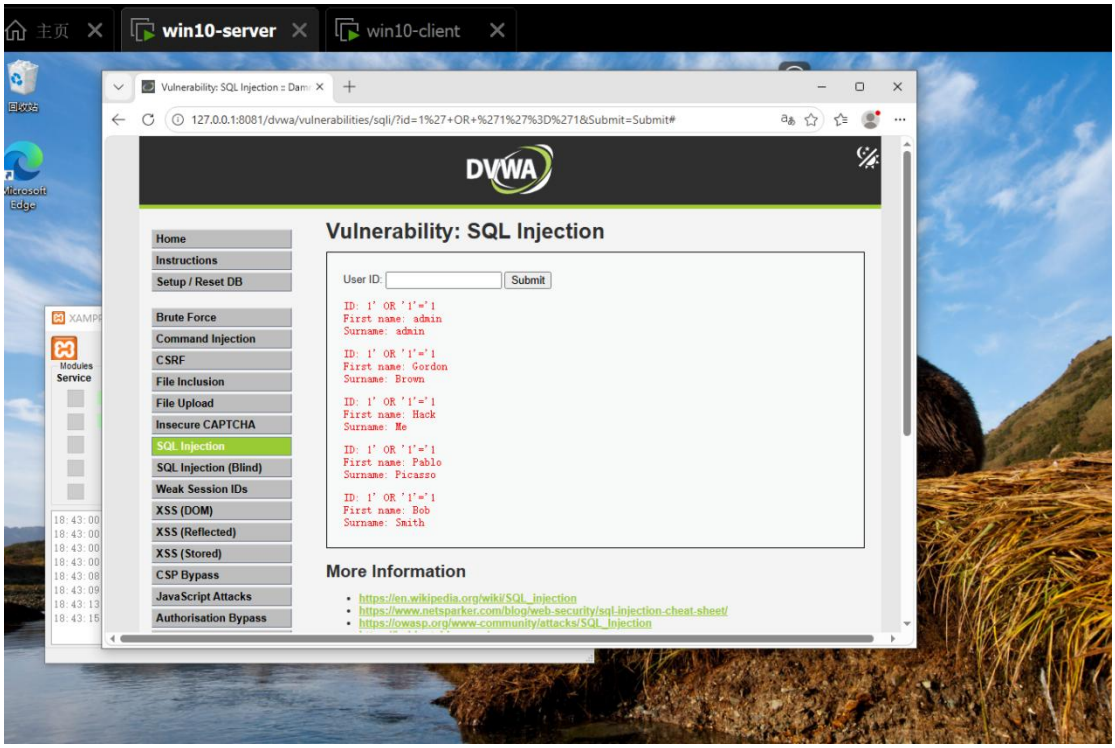
- 实验页面：DVWA 靶场的 SQL Injection 模块，安全级别设置为 Low。
- 实验目标：观察普通输入与异常输入在页面上的不同返回现象，验证该页面存在 SQL 注入漏洞，并分析漏洞成因。

2. 普通输入与异常输入的对比

类型	输入示例	返回结果
普通输入	1	返回 ID=1 的用户信息：First name：Admin，Surname：admin
异常输入（SQL 注入）	1' OR '1'='1	返回了数据库中所有用户的信息（不止一条）

3. 返回结果的差异





4. 为什么不像正常查询变化

• 正常查询的逻辑是“输入什么条件，就返回对应条件的数据”，结果的数量和内容都应该是可控的。

• 而这里的异常输入导致了查询逻辑被篡改，例如：

例如：1.输入 1’ OR ’1’=’1 后，返回的不是 ID=1 的单条数据，而是全表数据，这不

符合“按 ID 查询”的正常逻辑。
2.输入构造的 UNION 语句后，页面直接返回了数据库版本这类后台敏感信息，这不是一个正常的用户查询功能应该返回的内容。

• 这种差异说明，用户输入没有被后端安全过滤，而是被直接拼接到 SQL 语句中执行，因此产生了超出正常业务逻辑的结果，这是 SQL 注入漏洞的典型特征。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户在页面输入的内容，会通过前端 HTTP 请求（如 GET/POST）发送到后端；
后端代码直接将输入拼接到 SQL 语句的 WHERE 子句中，形成完整的查询结构；
再由数据库解析、执行该 SQL，并将结果返回页面。
完整链路：输入→拼接→解析→执行→回显或行为差异

2. 为什么根因是“结构被污染”而不是“字符本身危险”

单引号、OR、UNION 这些字符本身是合法的 SQL 语法元素，本身没有危险。
危险的是：后端未做过滤，让用户输入直接改变了原 SQL 的逻辑结构，把原本的查询条件篡改成了恶意逻辑。
例如：原 SQL 是 SELECT * FROM 用户 WHERE 账号='用户输入'，用户输入' OR '1'='1 后，SQL 结构从“单条件查询”变成了“恒成立条件查询”，是结构被污染，而非字符本身有害。

3. Low 与 Medium 的差异在哪里

• Low 级别：后端完全不做过滤，直接拼接用户输入，可直接用' OR '1'='1 这类语句完成注入，回显正常。

• Medium 级别：后端会做基础过滤（如转义单引号、替换特殊字符），但防护不彻底，攻击者可通过编码、宽字节注入等方式绕过，注入难度比 Low 更高，部分回显会被限制。

核心差异：Medium 做了基础防护但存在绕过空间，Low 无任何防护，直接拼接。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

• 普通 SQL 注入：数据库会直接返回查询结果到页面，攻击者可直接看到注入后的回显数据（如用户信息、数据库版本）。

• 盲注（Blind SQL Injection）：页面不直接返回查询结果，仅通过页面的“正常/异常”“时间延迟”等行为差异来判断注入结果，需要逐字符猜解数据。

核心差异：普通注入有直接回显，盲注无直接回显，依赖行为差异判断。

5. 为什么盲注和普通注入根因相同，但证据形式不同

• 根因相同：两者都是用户输入未过滤，直接拼接 SQL 导致结构被污染，本质都是 SQL 注入漏洞。

• 证据形式不同：普通注入的证据是页面直接回显非预期数据；盲注的证据是页面行为（如

响应时间、页面状态）出现非预期差异，没有直接数据回显。

任务 4：SQL 注入防护清单与解释

1. 参数化查询

- 解决什么问题：从根本上防止用户输入污染 SQL 语句结构，是防御 SQL 注入的核心手段。

- 控制哪一层：控制根因层，直接消除“用户输入拼接篡改 SQL 逻辑”的注入前提。

- 它不能替代什么：不能替代输入校验、最小权限等措施，也无法防御业务逻辑层面的漏洞。

- 说明：使用预编译语句，将 SQL 逻辑与用户输入严格分离，用户输入始终被当作普通数据处理，不会被解析成 SQL 指令。

2. 输入校验

- 解决什么问题：过滤恶意输入、缩小可接受的输入范围，降低注入攻击的成功率。

- 控制哪一层：控制暴露面层，减少可被利用的输入入口。

- 它不能替代什么：不能替代参数化查询，仅靠校验很容易被绕过（如编码、特殊字符变形）。

- 说明：对用户输入进行白名单校验，比如 ID 输入只允许数字、用户名只允许字母数字组合，拒绝不符合规则的输入。

3. 错误处理

- 解决什么问题：防止攻击者通过页面错误信息获取数据库结构、表名等敏感信息，避免为盲注提供线索。

- 控制哪一层：控制暴露面层，减少注入攻击的辅助信息。

- 它不能替代什么：不能阻止注入本身，只是隐藏了攻击的反馈信息

- 说明：后端统一返回通用错误提示（如“系统繁忙，请稍后再试”），不直接抛出数据库报错信息。

4. 最小权限

- 解决什么问题：即使发生 SQL 注入攻击，攻击者也无法执行高危操作（如删除表、读取敏感数据），降低攻击造成的损失。

- 控制哪一层：控制后果范围层，限制漏洞被利用后的影响范围。

- 它不能替代什么：不能阻止注入漏洞本身，只是限制了攻击的破坏力。

- 说明：数据库连接账号只分配业务必需的权限，禁止使用管理员账号连接 Web 应用，避免 DROP、ALTER 等高权限操作。

5.可补充其他合理措施

- 解决什么问题：识别并拦截常见的 SQL 注入攻击 payload，作为额外的防护层。

- 控制哪一层：控制暴露面层，在请求到达后端前拦截恶意请求。

- 它不能替代什么：不能替代参数化查询这类根本性防护，容易被复杂的变形注入绕过。

要点核对（全部覆盖最低要求）

收口项 1：SQL 注入因果链/防护关系图

1. **输入点**：用户在登录/查询页面输入内容，这是注入的起点。

2.**查询构造**：后端未对输入做处理，直接将用户输入拼接到 SQL 语句中，这是注入发生的关键环节。

3.**数据库解析/执行**：数据库执行被篡改的 SQL 语句，将恶意输入解析为 SQL 指令，执行非预期操作。

4.**结果差异/行为为证据**：页面返回全表数据、数据库版本等非预期内容，或出现页面报错、时间延迟等行为差异，这是注入成功的直接证据。

5.**根因判断**：注入的根本原因是“用户输入污染了 SQL 语句的结构”，而非输入中的特殊字符本身。

6.**防护控制点对应：**

参数化查询：作用于“查询构造”环节，从根源上防止输入污染 SQL 结构，直接消除注入根因。

输入校验：作用于“输入点”环节，过滤恶意输入，降低注入的暴露面。

最小权限：作用于“数据库解析/执行”环节，限制数据库账号权限，缩小注入攻击的后果范围。

收口项 2：学生自评表

1. **我本章最清楚的一点是什么？**

我最清楚的是 SQL 注入的根本成因：它不是由特殊字符本身造成的，而是用户输入未被过滤，

直接拼接篡改了 SQL 语句的结构。我也清晰掌握了“输入→拼接→解析→执行→回显/行为差异”这一完整的注入链路。

2.我最容易混淆的一点是什么？
我最容易混淆的是普通 SQL 注入与盲注的差异，尤其是如何通过页面行为（如响应时间、页面状态变化）来判断盲注的执行结果，以及如何构造不同的 payload 来实现逐字符猜解数据。

3.我本次作业中哪一段最需要教师复核？
我认为 SQL 注入因果链与防护关系图的部分最需要教师复核，尤其是参数化查询、输入校验等防护措施在因果链中的对应位置是否准确，是否完整体现了“防护措施控制根因、暴露面或后果范围”的逻辑。

4.如果重做一次，我会优先改哪一部分？
我会优先完善盲注的现象记录与分析，补充更多不同盲注场景（如布尔盲注、时间盲注）的 payload 和对应页面行为截图，让“无直接回显的证据链”更完整。
收口项 3：AI 输出审核与人工复核记录

收口项 3：AI 输出审核与人工复核记录

SQL 注入的根本成因是“结构被污染”，而非字符本身危险

参数化查询是防御 SQL 注入的核心手段，能直接消除注入根因

普通注入与盲注的核心差异在于“是否有直接数据回显”

输入校验、错误处理、最小权限的防护层级分别对应暴露面、暴露面、后果范围

3.哪些内容你发现 AI 输出不准确并进行了修正？

AI 最初将“DVWA Low 级别的防护逻辑”描述为“存在基础过滤”，我结合实际实验结果修正为“无任何过滤，直接拼接用户输入”

AI 最初混淆了“错误处理”的防护层级，我修正为“控制暴露面，减少攻击者获取的辅助信息，而非直接阻止注入”

AI 最初的因果链中缺少“结果差异→根因判断”的环节，我补充了该部分，完善了逻辑闭环

4.你最终如何确认结论和证据是一一对应的？

我将每一条结论都与 DVWA 实验中的实际现象（如输入 payload 后的页面回显、行为变化）进行了对应验证，确保结论不是凭空推导的。

我核对了所有防护措施的描述，确保它们与“控制根因/暴露面/后果范围”的作业要求一一匹配，没有概念混淆。

我对照作业的最低完成要求，逐条检查所有要点是否都有对应的分析和实验证据支撑